

The TypeScript Type Space

Think of a type as the SET of values it allows. Unions widen that set; intersections narrow it.

How wide is the set?

any

Every value, and it switches checking OFF. Avoid it.

unknown

Every value too, but you must narrow before using it. The safe any.

string

Every string. A normal, useful type.

'small' | 'large'

Exactly two values. Precise and self-documenting.

never

No value at all. The empty set.

Combining types

Union - `A | B` - either one
`string | number` accepts both, so you may only use what they have in COMMON until you narrow.

Intersection - `A & B` - both at once
`HasId & HasName` demands every member of each.
Fewer values qualify, but each has MORE properties.

type or interface?

interface - object shapes you may want to extend.
It can be reopened and merged by a later declaration.
type - everything else: unions, tuples, primitives, mapped and conditional types. Cannot be reopened.
For a plain object shape either is fine; be consistent.

TypeScript is STRUCTURAL: two types match when their shapes match, whatever they are called; nothing needs to declare that it implements an interface. And prefer inference: write the annotation where a value ENTERS your code (parameters, return types, external data) and let TypeScript work out the rest.